

Compound Components Pattern

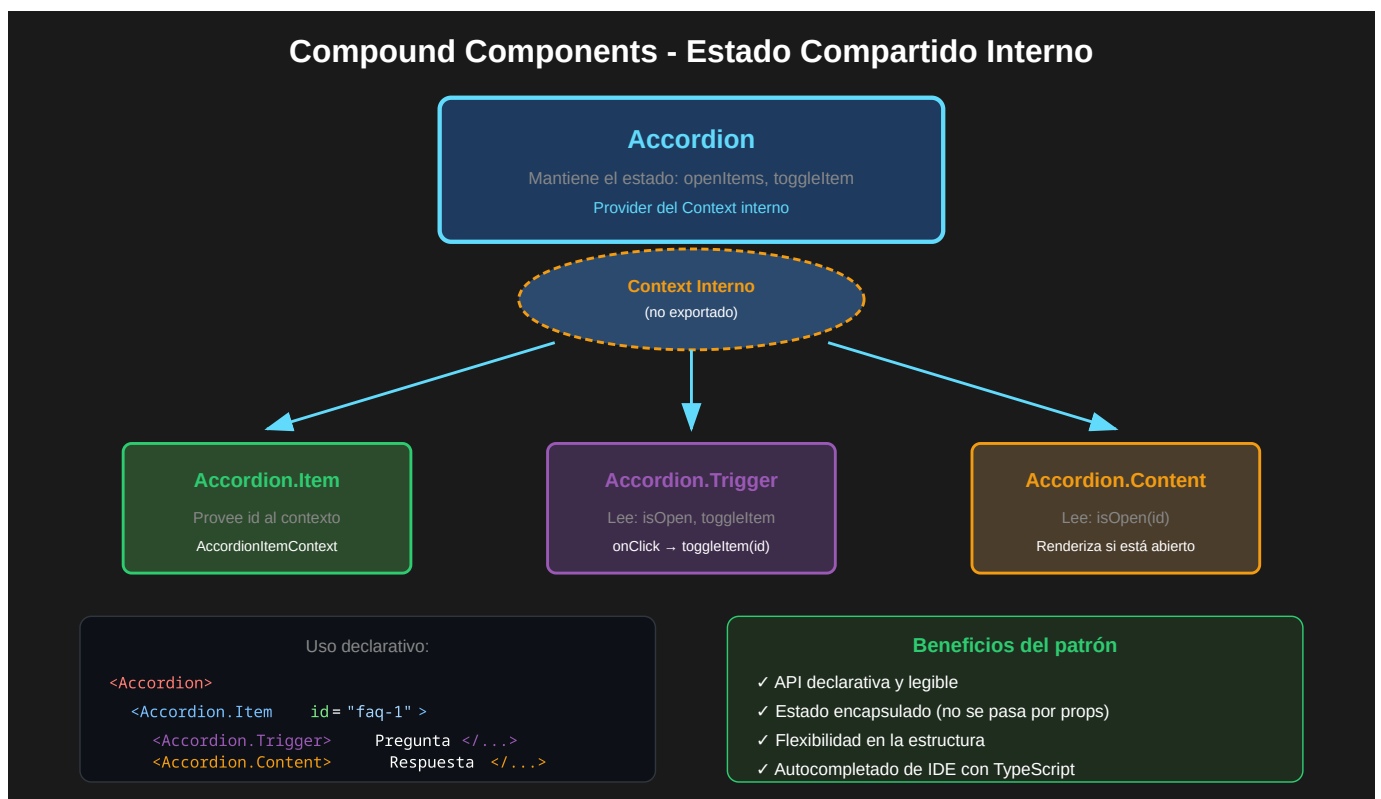
Objetivos

- Entender el patrón Compound Components
- Implementar componentes compuestos con Context interno
- Crear APIs declarativas y flexibles
- Tipar correctamente compound components en TypeScript

Contenido

1. ¿Qué son los Compound Components?

Los **Compound Components** son un conjunto de componentes que trabajan juntos para formar una funcionalidad completa. Comparten estado implícitamente sin que el usuario tenga que pasarlo manualmente.



Ejemplo: HTML nativo

HTML tiene compound components nativos:

```
<!-- select y option trabajan juntos -->
<select>
  <option value="a">Opción A</option>
  <option value="b">Opción B</option>
</select>

<!-- table, thead, tbody, tr, td trabajan juntos -->
<table>
  <thead>
    <tr>
```

```

        <th>Columna 1</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Dato 1</td>
      </tr>
    </tbody>
  </table>

```

El estado fluye implícitamente entre ellos. Vamos a crear lo mismo en React.

2. Compound Components Básico

2.1 Sin Context (limitado)

```

// Versión simple usando cloneElement
interface ToggleProps {
  children: ReactNode;
}

interface ToggleButtonProps {
  children: ReactNode;
  on?: boolean;
  toggle?: () => void;
}

interface ToggleContentProps {
  children: ReactNode;
  on?: boolean;
  when?: boolean;
}

const Toggle: React.FC<ToggleProps> & {
  Button: React.FC<ToggleButtonProps>;
  On: React.FC<ToggleContentProps>;
  Off: React.FC<ToggleContentProps>;
} = ({ children }) => {
  const [on, setOn] = useState(false);
  const toggle = () => setOn((prev) => !prev);

  // Clonar children para inyectar props
  const childrenWithProps = Children.map(children, (child) => {
    if (isValidElement(child)) {
      return cloneElement(child, { on, toggle });
    }
    return child;
  });

  return <div className="toggle">{childrenWithProps}</div>;
};

Toggle.Button = ({ children, on, toggle }) => (
  <button
    onClick={toggle}
    className={on ? 'active' : ''}>

```

```

    {children}
  </button>
);

Toggle.On = ({ children, on }) => (on ? <>{children}</> : null);

Toggle.Off = ({ children, on }) => (on ? null : <>{children}</>);

// Uso
<Toggle>
  <Toggle.Button>Activar</Toggle.Button>
  <Toggle.On>✅ Está encendido</Toggle.On>
  <Toggle.Off>❌ Está apagado</Toggle.Off>
</Toggle>;

```

⚠️ **Limitación:** `cloneElement` solo funciona con hijos directos. Si envuelves un componente, pierde los props.

3. Compound Components con Context

La forma correcta y escalable es usar **Context interno**:

```

import { createContext, useContext, useState, ReactNode } from 'react';

// =====
// 1. DEFINIR TIPOS
// =====

// QUÉ: Interface del contexto interno del Accordion
// PARA: Definir qué datos y funciones comparten los subcomponentes
// IMPACTO: Los subcomponentes pueden comunicarse sin props explícitos
interface AccordionContextValue {
  openItems: Set<string>;
  toggleItem: (id: string) => void;
  isOpen: (id: string) => boolean;
}

interface AccordionProps {
  children: ReactNode;
  defaultOpen?: string[];
  allowMultiple?: boolean;
}

interface AccordionItemProps {
  children: ReactNode;
  id: string;
}

interface AccordionTriggerProps {
  children: ReactNode;
}

interface AccordionContentProps {
  children: ReactNode;
}

// =====

```

```
// 2. CREAR CONTEXTO INTERNO
// =====

// QUÉ: Contexto interno que NO se exporta al usuario
// PARA: Comunicación entre componentes del mismo grupo sin exponer detalles
// IMPACTO: API limpia para el usuario, estado compartido internamente
const AccordionContext = createContext<AccordionContextValue | null>(null);
const AccordionItemContext = createContext<string | null>(null);

// QUÉ: Hooks internos con validación de contexto
// PARA: Garantizar que subcomponentes se usen dentro del componente padre
// IMPACTO: Errores claros si se usa AccordionTrigger fuera de Accordion
const useAccordionContext = () => {
  const context = useContext(AccordionContext);
  if (!context) {
    throw new Error('Accordion components must be used within an Accordion');
  }
  return context;
};

const useAccordionItemContext = () => {
  const id = useContext(AccordionItemContext);
  if (!id) {
    throw new Error(
      'AccordionTrigger/Content must be used within AccordionItem',
    );
  }
  return id;
};

// =====
// 3. COMPONENTE PRINCIPAL
// =====

// QUÉ: Componente principal con subcomponentes asignados como propiedades
// PARA: Crear API declarativa tipo <Accordion.Item> en lugar de props complejos
// IMPACTO: Código legible, autocompletado de IDE, flexibilidad de estructura
const Accordion: React.FC<AccordionProps> & {
  Item: React.FC<AccordionItemProps>;
  Trigger: React.FC<AccordionTriggerProps>;
  Content: React.FC<AccordionContentProps>;
} = ({ children, defaultOpen = [], allowMultiple = false }) => {
  const [openItems, setOpenItems] = useState<Set<string>>(new Set(defaultOpen));

  const toggleItem = (id: string) => {
    setOpenItems((prev) => {
      const next = new Set(prev);

      if (next.has(id)) {
        next.delete(id);
      } else {
        if (!allowMultiple) {
          next.clear();
        }
        next.add(id);
      }
    });
  };
};
```

```

        return next;
    });
};

const isOpen = (id: string) => openItems.has(id);

return (
    <AccordionContext.Provider value={{ openItems, toggleItem, isOpen }}>
        <div className="accordion">{children}</div>
    </AccordionContext.Provider>
);
};

// =====
// 4. COMPONENTES HIJOS
// =====

const AccordionItem: React.FC<AccordionItemProps> = ({ children, id }) => {
    const { isOpen } = useAccordionContext();

    return (
        <AccordionItemContext.Provider value={id}>
            <div className={`accordion-item ${isOpen(id) ? 'open' : ''}`>
                {children}
            </div>
        </AccordionItemContext.Provider>
    );
};

const AccordionTrigger: React.FC<AccordionTriggerProps> = ({ children }) => {
    const { toggleItem, isOpen } = useAccordionContext();
    const id = useAccordionItemContext();

    return (
        <button
            className="accordion-trigger"
            onClick={() => toggleItem(id)}
            aria-expanded={isOpen(id)}>
            {children}
            <span className="icon">{isOpen(id) ? '-' : '+'}</span>
        </button>
    );
};

const AccordionContent: React.FC<AccordionContentProps> = ({ children }) => {
    const { isOpen } = useAccordionContext();
    const id = useAccordionItemContext();

    if (!isOpen(id)) return null;

    return <div className="accordion-content">{children}</div>;
};

// Asignar subcomponentes
Accordion.Item = AccordionItem;

```

```

Accordion.Trigger = AccordionTrigger;
Accordion.Content = AccordionContent;

// =====
// 5. USO
// =====

const FAQ: React.FC = () => (
  <Accordion
    defaultOpen={['faq-1']}
    allowMultiple={false}>
    <Accordion.Item id="faq-1">
      <Accordion.Trigger>¿Qué es React?</Accordion.Trigger>
      <Accordion.Content>
        React es una biblioteca de JavaScript para construir interfaces de
        usuario.
      </Accordion.Content>
    </Accordion.Item>

    <Accordion.Item id="faq-2">
      <Accordion.Trigger>¿Qué es TypeScript?</Accordion.Trigger>
      <Accordion.Content>
        TypeScript es un superset de JavaScript que añade tipado estático.
      </Accordion.Content>
    </Accordion.Item>

    <Accordion.Item id="faq-3">
      <Accordion.Trigger>¿Por qué usar ambos?</Accordion.Trigger>
      <Accordion.Content>
        La combinación proporciona una experiencia de desarrollo superior con
        mejor autocompletado, detección de errores y documentación inline.
      </Accordion.Content>
    </Accordion.Item>
  </Accordion>
);

```

4. Compound Components Avanzado: Menu

```

import {
  createContext,
  useContext,
  useState,
  useRef,
  useEffect,
  ReactNode,
} from 'react';

// =====
// TIPOS
// =====

interface MenuContextValue {
  isOpen: boolean;
  open: () => void;
  close: () => void;
}

```

```

    toggle: () => void;
    activeIndex: number;
    setActiveIndex: (index: number) => void;
  }

  // =====
  // CONTEXTO
  // =====

  const MenuContext = createContext<MenuContextValue | null>(null);

  const useMenuContext = () => {
    const context = useContext(MenuContext);
    if (!context) {
      throw new Error('Menu components must be used within a Menu');
    }
    return context;
  };

  // =====
  // COMPONENTES
  // =====

  interface MenuProps {
    children: ReactNode;
  }

  const Menu: React.FC<MenuProps> & {
    Trigger: React.FC<{ children: ReactNode }>;
    List: React.FC<{ children: ReactNode }>;
    Item: React.FC<{
      children: ReactNode;
      onClick?: () => void;
      disabled?: boolean;
    }>;
    Divider: React.FC;
  } = ({ children }) => {
    const [isOpen, setIsOpen] = useState(false);
    const [activeIndex, setActiveIndex] = useState(-1);
    const menuRef = useRef<HTMLDivElement>(null);

    const open = () => setIsOpen(true);
    const close = () => {
      setIsOpen(false);
      setActiveIndex(-1);
    };

    const toggle = () => (isOpen ? close() : open());

    // Cerrar al hacer click fuera
    useEffect(() => {
      const handleClickOutside = (e: MouseEvent) => {
        if (menuRef.current && !menuRef.current.contains(e.target as Node)) {
          close();
        }
      };
    });
  };

```

```

    if (isOpen) {
      document.addEventListener('mousedown', handleClickOutside);
    }

    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, [isOpen]);

// Cerrar con Escape
useEffect(() => {
  const handleEscape = (e: KeyboardEvent) => {
    if (e.key === 'Escape') close();
  };

  if (isOpen) {
    document.addEventListener('keydown', handleEscape);
  }

  return () => document.removeEventListener('keydown', handleEscape);
}, [isOpen]);

return (
  <MenuContext.Provider
    value={{ isOpen, open, close, toggle, activeIndex, setActiveIndex }}>
    <div
      className="menu"
      ref={menuRef}>
      {children}
    </div>
  </MenuContext.Provider>
);
};

const MenuTrigger: React.FC<{ children: ReactNode }> = ({ children }) => {
  const { toggle, isOpen } = useMenuContext();

  return (
    <button
      className="menu-trigger"
      onClick={toggle}
      aria-expanded={isOpen}
      aria-haspopup="menu">
      {children}
    </button>
  );
};

const MenuList: React.FC<{ children: ReactNode }> = ({ children }) => {
  const { isOpen } = useMenuContext();

  if (!isOpen) return null;

  return (
    <ul
      className="menu-list"
      role="menu">
      {children}
    </ul>
  );
};

```



```

    </ul>
  );
};

const MenuItem: React.FC<{
  children: ReactNode;
  onClick?: () => void;
  disabled?: boolean;
}> = ({ children, onClick, disabled = false }) => {
  const { close } = useMenuContext();

  const handleClick = () => {
    if (disabled) return;
    onClick?.();
    close();
  };

  return (
    <li
      className={`menu-item ${disabled ? 'disabled' : ''}`}
      role="menuitem"
      onClick={handleClick}
      aria-disabled={disabled}>
      {children}
    </li>
  );
};

const MenuDivider: React.FC = () => (
  <li
    className="menu-divider"
    role="separator"
  />
);

// Asignar subcomponentes
Menu.Trigger = MenuTrigger;
Menu.List = MenuList;
Menu.Item = MenuItem;
Menu.Divider = MenuDivider;

// =====
// USO
// =====

const UserMenu: React.FC = () => (
  <Menu>
    <Menu.Trigger>
      <span>👤 Mi Cuenta</span>
    </Menu.Trigger>
    <Menu.List>
      <Menu.Item onClick={() => console.log('Perfil')}>Ver Perfil</Menu.Item>
      <Menu.Item onClick={() => console.log('Configuración')}>
        Configuración
      </Menu.Item>
      <Menu.Divider />
    </Menu.List>
  </Menu>
);

```

```

    <Menu.Item onClick={() => console.log('Cerrar sesión')}>
      Cerrar Sesión
    </Menu.Item>
  </Menu.List>
</Menu>
);

```

5. Compound Components con Estado Controlado

A veces quieres que el padre pueda controlar el estado:

```

interface TabsProps {
  children: ReactNode;
  // Modo no controlado
  defaultValue?: string;
  // Modo controlado
  value?: string;
  onChange?: (value: string) => void;
}

interface TabsContextValue {
  activeTab: string;
  setActiveTab: (value: string) => void;
}

const TabsContext = createContext<TabsContextValue | null>(null);

const useTabs = () => {
  const context = useContext(TabsContext);
  if (!context) {
    throw new Error('Tabs components must be used within Tabs');
  }
  return context;
};

const Tabs: React.FC<TabsProps> & {
  List: React.FC<{ children: ReactNode }>;
  Tab: React.FC<{ value: string; children: ReactNode }>;
  Panel: React.FC<{ value: string; children: ReactNode }>;
} = ({ children, defaultValue = '', value, onChange }) => {
  // Estado interno para modo no controlado
  const [internalValue, setInternalValue] = useState(defaultValue);

  // Determinar si es controlado o no
  const isControlled = value !== undefined;
  const activeTab = isControlled ? value : internalValue;

  const setActiveTab = (newValue: string) => {
    // Si es controlado, solo notificar al padre
    if (isControlled) {
      onChange?.(newValue);
    } else {
      // Si no es controlado, actualizar estado interno
      setInternalValue(newValue);
      onChange?.(newValue); // También notificar si hay callback
    }
  };

```

```

    }
  };

  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
};

const TabsList: React.FC<{ children: ReactNode }> = ({ children }) => (
  <div
    className="tabs-list"
    role="tablist">
    {children}
  </div>
);

const Tab: React.FC<{ value: string; children: ReactNode }> = ({
  value,
  children,
}) => {
  const { activeTab, setActiveTab } = useTabs();
  const isActive = activeTab === value;

  return (
    <button
      className={`tab ${isActive ? 'active' : ''}`}
      role="tab"
      aria-selected={isActive}
      onClick={() => setActiveTab(value)}>
      {children}
    </button>
  );
};

const TabPanel: React.FC<{ value: string; children: ReactNode }> = ({
  value,
  children,
}) => {
  const { activeTab } = useTabs();

  if (activeTab !== value) return null;

  return (
    <div
      className="tab-panel"
      role="tabpanel">
      {children}
    </div>
  );
};

Tabs.List = TabsList;
Tabs.Tab = Tab;
Tabs.Panel = TabPanel;

```

```
// =====
// USO - Modo No Controlado
// =====

const UncontrolledExample: React.FC = () => (
  <Tabs
    defaultValue="tab1"
    onChange={(v) => console.log('Tab changed:', v)}>
    <Tabs.List>
      <Tabs.Tab value="tab1">Tab 1</Tabs.Tab>
      <Tabs.Tab value="tab2">Tab 2</Tabs.Tab>
    </Tabs.List>
    <Tabs.Panel value="tab1">Contenido 1</Tabs.Panel>
    <Tabs.Panel value="tab2">Contenido 2</Tabs.Panel>
  </Tabs>
);

// =====
// USO - Modo Controlado
// =====

const ControlledExample: React.FC = () => {
  const [activeTab, setActiveTab] = useState('tab1');

  return (
    <>
      <p>Tab activo: {activeTab}</p>
      <button onClick={() => setActiveTab('tab1')}>Ir a Tab 1</button>
      <button onClick={() => setActiveTab('tab2')}>Ir a Tab 2</button>

      <Tabs
        value={activeTab}
        onChange={setActiveTab}>
        <Tabs.List>
          <Tabs.Tab value="tab1">Tab 1</Tabs.Tab>
          <Tabs.Tab value="tab2">Tab 2</Tabs.Tab>
        </Tabs.List>
        <Tabs.Panel value="tab1">Contenido 1</Tabs.Panel>
        <Tabs.Panel value="tab2">Contenido 2</Tabs.Panel>
      </Tabs>
    </>
  );
};
```

6. Tipado Estricto con displayName

Para TypeScript más estricto y mejor debugging:

```
// Definir el tipo del compound component completo
interface CardComponent extends React.FC<{
  children: ReactNode;
  className?: string;
}> {
  Header: React.FC<{ children: ReactNode }>;
}
```

```

    Body: React.FC<{ children: ReactNode }>;
    Footer: React.FC<{ children: ReactNode }>;
    Image: React.FC<{ src: string; alt: string }>;
  }

  // Crear componentes individuales
  const CardHeader: React.FC<{ children: ReactNode }> = ({ children }) => (
    <header className="card-header">{children}</header>
  );
  CardHeader.displayName = 'Card.Header';

  const CardBody: React.FC<{ children: ReactNode }> = ({ children }) => (
    <main className="card-body">{children}</main>
  );
  CardBody.displayName = 'Card.Body';

  const CardFooter: React.FC<{ children: ReactNode }> = ({ children }) => (
    <footer className="card-footer">{children}</footer>
  );
  CardFooter.displayName = 'Card.Footer';

  const CardImage: React.FC<{ src: string; alt: string }> = ({ src, alt }) => (
    <img
      className="card-image"
      src={src}
      alt={alt}
    />
  );
  CardImage.displayName = 'Card.Image';

  // Componente principal
  const Card: CardComponent = ({ children, className = '' }) => (
    <article className={`card ${className}`}>{children}</article>
  );
  Card.displayName = 'Card';

  // Asignar subcomponentes
  Card.Header = CardHeader;
  Card.Body = CardBody;
  Card.Footer = CardFooter;
  Card.Image = CardImage;

  // Uso con autocompletado perfecto
  <Card className="product-card">
    <Card.Image
      src="/product.jpg"
      alt="Producto"
    />
    <Card.Header>
      <h3>Nombre del Producto</h3>
    </Card.Header>
    <Card.Body>
      <p>Descripción del producto...</p>
    </Card.Body>
    <Card.Footer>
      <button>Comprar</button>
    </Card.Footer>
  </Card>

```

```
</Card.Footer>
</Card>;
```

7. Beneficios del Patrón

Beneficio	Descripción
API Declarativa	El código es legible y auto-documentado
Flexibilidad	El usuario decide la estructura
Encapsulación	Estado interno oculto del consumidor
Reutilización	Subcomponentes reutilizables
TypeScript	Autocompletado y type safety

✓ Checklist de Verificación

- ☐ Entiendo qué son los Compound Components
- ☐ Sé crear contexto interno para compartir estado
- ☐ Puedo implementar hooks seguros que validan contexto
- ☐ Entiendo la diferencia entre modo controlado y no controlado
- ☐ Sé usar displayName para mejor debugging
- ☐ Puedo crear APIs declarativas y flexibles

Recursos Adicionales

- [Kent C. Dodds - Compound Components](#)
- [Patterns.dev - Compound Pattern](#)
- [Headless UI](#) - Ejemplos de compound components

Siguiente: [Context API](#)